

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: 2026-07-24 | **Scope:** shende-shweta/FSDKC (main) — PHP 8.3 / Laravel 12 backend · React 19 / TypeScript / Vite frontend · Node/Express dev-api · MariaDB + MongoDB

Executive Summary

EXECUTIVE SUMMARY

Klearcom is a modular monolith (Discovery IVR + Connect TFN) with a thin Laravel 12 API, a React 19 SPA, and a parallel Express dev-api that re-implements the same workflows. Controllers are short by LOC but fat by responsibility: Eloquent and KPI math live in HTTP handlers, Modules folders are documentation stubs only, and there are zero repository classes. The dominant risk is change amplification from duplicated business logic across Laravel, Express, and the SPA, plus shared Mongo collections and cross-domain dashboard/legacy reports that erase bounded-context ownership. Frontend has a usable api/client and React Query, but legacy class/widget patterns and hard-coded endpoint strings in pages still couple UI to backend shapes.

7

CONTROLLERS / HANDLERS

4

MODELS / ENTITIES

2

SERVICE CLASSES FOUND

0

REPOSITORY CLASSES
FOUND

Layers covered: Backend Laravel (backend/app/** , 6 API controllers + 2 services + 4 models + Legacy mapper) · Frontend React (frontend/src/** , 9 page/component files + api/hooks/store) · Dev API Express (dev-api/src/** , 6 modules). File counts analyzed: ~18 backend PHP application files, ~15 frontend TS/TSX/JSX source files, ~6 Node API source files.

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Domain Boundary Violations (H8), Shared DB Coupling (H9), and Dual-API duplication (H10).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC (Laravel avg); Express server.js 224 LOC	MODERATE

H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 handler methods with direct model/store access	HIGH
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	35+ Eloquent/store/Mongo access points; 0 repositories	HIGH
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (<code>LegacyDataMapper</code> , shared <code>buildTree</code> helpers)	MODE
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~38% of queries outside controllers	HIGH
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest <code>server.js</code> 224 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8 cross-domain access points	HIGH
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	37.5% (3/8 stores shared via Mongo <code>module</code>)	HIGH
H10	Dual API Duplication (additional)	Duplicated workflows across Laravel ↔ Express	0	1–3	>3	5+ workflows duplicated	HIGH
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components with hard-coded paths	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest ConnectPage 208 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; small Zustand UI store	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (<code>LegacyMonitorPoller</code> , <code>LegacyDashboardWidget</code>)	MODE

H10 KPI thresholds (additional): duplicated domain workflows across parallel stacks — Good = 0 · Moderate = 1–3 · High Risk = >3.

1.2 Hotspot-by-Hotspot Evidence

H1. Fat Controllers HIGH

Benchmark: Avg LOC per controller = 62 (Laravel) · worst file `dev-api/src/server.js` = 224 LOC → falls in the **Moderate** band (Good <150 · Moderate 150–300 · High Risk >300; worst-wins via Express handler size + business logic density).

What to check: Business logic inside controllers/handlers — validation, KPI math, tree building, reachability formulas — instead of Application/Domain Services.

Evidence: Six Laravel API controllers average ~62 LOC, but several embed domain rules. Express packs the entire HTTP surface into one file.

`backend/app/Http/Controllers/Api/ConnectController.php:65-88` — reachability KPI computed inside the handler:

```
$recentChecks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(20)
    ->get();

$successRate = $recentChecks->count() > 0
    ? ($recentChecks->where('reachable', true)->count() / $recentChecks-
    >count()) * 100
    : 100;

$computedStatus = $successRate < 90 ? 'alert' : 'active';
```

This qualifies because alert thresholds and success-rate math are domain rules, not HTTP mapping.

`backend/app/Http/Controllers/Api/LegacyReportController.php:1-20` — explicitly documents itself as a fat controller and owns carrier filtering, KPI math, and tree building:

```
/**
 * Fat controller – business logic, DB queries, and KPI math live here (anti-
 * pattern).
 */
class LegacyReportController extends Controller
{
    public function carrierSummary(Request $request): JsonResponse
    {
        $filters = $request->all();
        extract($filters);
```

`dev-api/src/server.js` (224 LOC) — every Discovery/Connect/Dashboard route, SSE streaming, and KPI aggregation live in one Express handler module (duplicate of Laravel controllers).

Why it matters here: Reachability logic is already duplicated in `RealTimeTestService` and `dev-api/src/realtime.js`. Changing the 90% alert threshold requires coordinated edits across Laravel controllers, Laravel services, and Express — the first regression will be inconsistent dashboard vs. check-history status.

Recommended approach:

1. Extract `ReachabilityCalculator` / `ConnectApplicationService` from `ConnectController::checks` and `RealTimeTestService::runConnectTest`.
2. Move `buildTree` + IVR depth stats from `DiscoveryController` / `LegacyReportController` into a `DiscoveryTreeService`.
3. Split `dev-api/src/server.js` into route modules that call shared application services (or retire Express once Laravel is the single API).

No files matched `backend/app/Http/Controllers/**/*.php` containing `(ConnectMonitor::|DiscoveryJob::|ConnectCheckResult::|DiscoveryNode::|extract|(|buildTree|reachability|successRate))`.

No files matched `dev-api/src/server.js` containing `app\.(get|post)\(''/api/`.

H2. Missing Service Layer CRITICAL

Benchmark: `Controllers/handlers directly accessing models/store = 25` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Business rules spread across controllers/utilities with no dedicated application-service tier reusable from HTTP, CLI, and jobs.

Evidence: Only two services exist (`MongoService`, `RealTimeTestService`); CRUD and reporting bypass them.

`backend/app/Http/Controllers/Api/DashboardController.php:12-36` — KPI workflow entirely in the controller with direct Eloquent:

```
public function kpis(): JsonResponse
{
    $discoveryTotal = DiscoveryJob::count();
    $discoveryCompleted = DiscoveryJob::where('status', 'completed')->count();
    $connectMonitors = ConnectMonitor::count();
    $avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
    $alerts = ConnectMonitor::where('status', 'alert')->count();
```

`backend/app/Http/Controllers/Api/DiscoveryController.php:24-45` — create-job workflow is controller-owned:

```
$job = DiscoveryJob::create([
    ...$validated,
    'status' => 'pending',
    'languages' => $validated['languages'] ?? ['en'],
]);
```

`dev-api/src/server.js` dashboard + discovery POST handlers mutate `store` arrays inline (same workflows, second stack). Across both stacks, **25** handler methods touch models/store directly (13 Laravel + 12 Express).

Why it matters here: `npm run dev` runs Express while Docker runs Laravel — the same UI hits different service implementations. Without an Application Service contract, CLI seeders, SSE jobs, and HTTP will keep drifting.

Recommended approach:

1. Introduce `DashboardKpiService`, `DiscoveryJobService`, `ConnectMonitorService`.
2. Thin controllers to validate → service → JSON resource.
3. Point Express (if kept) at the same service contracts via HTTP to Laravel or a shared package.

No files matched `backend/app/Http/Controllers/**/*.php` containing `(DiscoveryJob::|ConnectMonitor::|ConnectCheckResult::|DiscoveryNode::)`.

No files matched `dev-api/src/**/*.js` containing `store\.(discoveryJobs|connectMonitors|connectChecks|discoveryNodes)`.

H3. Missing Repository Pattern CRITICAL

Benchmark: `Direct DB/ORM access points outside repositories = 35+; Repository classes = 0` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Direct Eloquent / collection / in-memory store access scattered through controllers and services with no repository abstraction.

Evidence: No `*Repository` classes under `backend/app`. Models are queried from controllers and `RealTimeTestService`. Mongo access is concentrated in `MongoService` (closest thing to a repository) but Eloquent has no counterpart.

`backend/app/Services/RealTimeTestService.php:100-120` — persistence mixed into test orchestration:

```
ConnectCheckResult::create([
    'connect_monitor_id' => $monitorId,
    'reachable' => $reachable,
    'latency_ms' => $latency,
    ...
]);

$recent = ConnectCheckResult::where('connect_monitor_id', $monitorId)
    ->orderByDesc('checked_at')->limit(20)->get();
```

`backend/app/Http/Controllers/Api/LegacyReportController.php:28-45` — nested Eloquent loops (N+1 style) in the controller:

```
foreach ($monitors as $monitor) {
    $recent = ConnectCheckResult::where('connect_monitor_id', $monitor->id)
        ->orderByDesc('checked_at')
        ->limit(20)
        ->get();
```

Why it matters here: Schema lives in manual `docker/mariadb/init.sql`. Without repositories, swapping in-memory `dev-api` store for MariaDB (or testing without DB) requires rewriting every controller and service call site.

Recommended approach:

1. Add `DiscoveryJobRepository`, `ConnectMonitorRepository`, `ConnectCheckResultRepository` interfaces + Eloquent implementations.
2. Keep `MongoService` as the Mongo repository (or rename to `TranscriptRepository` / `TestEventRepository`).
3. Bind interfaces in `AppServiceProvider`.

No files matched `backend/app/**/*.php` containing `(::create\(\)::where\(\)::findOrFail\(\)::orderByDesc\(\)::count\(\)::avg\()`.

H4. Circular Dependencies LOW

Benchmark: `Dependency cycles = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Modules/packages importing each other in a cycle.

Evidence: Not observed — `backend/app/Modules/Discovery` and `Modules/Connect` contain only `AGENTS.md` (no PHP packages to cycle). Laravel controllers → services → models is a one-way graph. Frontend imports are acyclic (pages → components/hooks/api).

H5. Shared Utility Abuse MEDIUM

Benchmark: `Utility files holding business logic = 2` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Large common/helpers/utils holding business logic used everywhere.

Evidence:

`backend/app/Legacy/LegacyDataMapper.php:10-24` — domain report shaping via unsafe `extract()` :

```
public function mapReportRow(array $row): array
{
    extract($row, EXTR_SKIP);

    return [
        'label' => $name ?? 'Unknown',
        'metric' => $reachability_pct ?? 0,
        'region' => $country_code ?? 'N/A',
        'source' => 'legacy_extract_mapper',
    ];
}
```

`buildTree` is copy-pasted in `DiscoveryController`, `LegacyReportController`, and `dev-api/src/store.js` — a shared “utility” of domain tree logic with no single owner.

Why it matters here: `LegacyReportController` instantiates `new LegacyDataMapper()` (no DI). Tree shape changes for the SPA `IvrTree` require three backend edits plus the Express store.

Recommended approach: Replace `LegacyDataMapper` with typed DTOs; centralize `IvrTreeBuilder` as a Discovery domain service; delete duplicate methods.

No files matched `backend/app/**/*.php` containing `(extract\(|function buildTree|LegacyDataMapper)`.

No files matched `dev-api/src/**/*.js` containing `export function buildTree`.

H6. Direct SQL in Controllers CRITICAL

Benchmark: `ORM/repository compliance ≈ 38%` (queries kept out of controllers) → falls in the **High Risk** band (Good >90% · Moderate 60–90% · High Risk <60%). No raw SQL strings observed; Eloquent/Mongo/in-memory access still sits in handlers.

What to check: Raw queries or ORM calls embedded in controllers/handlers instead of repositories.

Evidence: Controllers issue the majority of Eloquent queries. Example from

`ConnectController::index/store/show/checks` — all direct model access. `DashboardController` issues 6+ aggregate queries. Express `server.js` diagnostics route hits Mongo collections inline:

```
const data = await getDb()
  .collection('call_diagnostics')
  .find({ module: req.params.module, reference_id:
    Number(req.params.referenceId) })
```

Counted ~40 distinct query call sites; ~15 live in `MongoService` / `RealTimeTestService` → ~38% compliance.

Why it matters here: Manual schema in `init.sql` means column renames (e.g. `reachability_pct`) touch every controller query site and the Express store shape simultaneously.

Recommended approach: Move all reads/writes behind repositories; ban Eloquent facades in `Http/Controllers` via PHPCS/PHPStan rules.

No files matched `backend/app/Http/Controllers/**/*.php` containing `(DB::|::where|(|::create|(|::findOrFail|(|::orderByDesc|(|collection|()`.

No files matched `dev-api/src/server.js` containing `(store.|getDb|(|)|collection|()`.

H7. God Classes LOW

Benchmark: `Classes/files >1000 LOC = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single classes/files handling many unrelated responsibilities at extreme size.

Evidence: Not observed — largest application files are `dev-api/src/server.js` (224 LOC), `MongoService` (132 LOC), `ConnectPage` (208 LOC). Size is healthy; cohesion issues are covered under H1/H2/H10 rather than raw LOC.

H8. Domain Boundary Violations CRITICAL

Benchmark: `Cross-domain access points = 8` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Code in one business area directly reading/writing another area's data or models; empty module boundaries.

Evidence: Declared modules exist but hold no code:

- `backend/app/Modules/Discovery/AGENTS.md` only
- `backend/app/Modules/Connect/AGENTS.md` only
- Same pattern under `frontend/src/modules/*`

Cross-domain access points measured:

1. `DashboardController` → `DiscoveryJob` + `ConnectMonitor`
2. `LegacyReportController` → Connect + Discovery models
3. `RealTimeTestService` → both domains
4. `MongoService` → shared collections keyed by `module` string
5. `LegacyDashboardWidget` → `/discovery/jobs` + `/connect/monitors` + KPIs
6. Express `server.js` dashboard KPI mixes both stores
7. `StreamController` shared SSE for both domains
8. Frontend pages share one `api/client` with no module ACL

`backend/app/Http/Controllers/Api/DashboardController.php` mixes Discovery and Connect aggregates in one response. `LegacyReportController` owns both carrier (Connect) and IVR depth (Discovery) reports.

Why it matters here: The README presents Discovery and Connect as extractable modules, but runtime code has no ownership boundary — extracting Connect would drag Dashboard, Legacy reports, Mongo helpers, and RealTimeTestService with it.

Recommended approach:

1. Move controllers/services/models under real `Modules/Discovery` and `Modules/Connect` namespaces.
2. Dashboard becomes a separate Reporting context that consumes published read models/DTOs only.
3. Add Anti-Corruption mapping for legacy report endpoints.

No files matched `backend/app/Http/Controllers/Api/{Dashboard,LegacyReport}Controller.php` containing `(DiscoveryJob|ConnectMonitor|DiscoveryNode|ConnectCheckResult)`.

No files matched `frontend/src/pages/LegacyDashboardWidget.tsx` containing `(discovery/jobs|connect/monitors|dashboard/kpis)`.

H9. Shared Database Coupling CRITICAL

Benchmark: `Tables/collections shared across domains = 37.5% (3 of 8)` → falls in the **High Risk** band (Good <10% · Moderate 10–30% · High Risk >30%).

What to check: Multiple business domains reading/writing the same tables/collections directly.

Evidence: MariaDB tables in `docker/mariadb/init.sql` are domain-split (`discovery_*` vs `connect_*`), but Mongo collections `transcripts`, `test_events`, and `call_diagnostics` are shared by both modules via a `module` discriminator — written from `MongoService` and `dev-api/src/mongo.js` / `realtime.js`.

```
// MongoService – single collection for all modules
$this->transcripts->insertOne([
  'module' => $module,
  'reference_id' => $referenceId,
  'payload' => $payload,
```

Relational schema is one shared database with no schema ownership or separate credentials per domain.

Why it matters here: Index/TTL changes for Connect diagnostics risk Discovery transcript performance. There is no ACL when Discovery code reads Connect-shaped Mongo documents.

Recommended approach: Per-context Mongo collections (or DB names); Reporting reads via ACL/DTO; eventually separate MariaDB schemas or databases per BC.

No files matched `backend/app/Services/MongoService.php` containing `(transcripts|test_events|call_diagnostics|selectCollection|collection|)`.

No files matched `dev-api/src/mongo.js` containing `(transcripts|test_events|call_diagnostics)`.

H10. Dual API Duplication (additional) CRITICAL

Benchmark: `Duplicated workflows across Laravel ↔ Express = 5+` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Parallel stacks implementing the same domain workflows without a single source of truth or ACL.

Evidence: README advertises Laravel for Docker and Express for `npm run dev`. Duplicated workflows observed:

1. Discovery test orchestration — `RealTimeTestService::runDiscoveryTest` ↔ `realtime.js::runDiscoveryTest`
2. Connect reachability test — `runConnectTest` in both stacks
3. Dashboard KPI formula — `DashboardController` ↔ `server.js /api/dashboard/kpis`
4. IVR `buildTree` — PHP controllers ↔ `store.js`
5. Reachability % / alert threshold — Controllers + both realtime modules

Frontend `VITE_API_URL` defaults to `http://localhost:8080/api` (Express), so local UI never exercises Laravel application services.

Why it matters here: This is the primary change-amplification hotspot — every domain fix ships twice or ships inconsistently.

Recommended approach: Pick one API runtime for local+prod (prefer Laravel); make Express a thin proxy or delete it; share OpenAPI contract with the SPA.

No files matched `{backend/app/**/*.*.php,dev-api/src/**/*.*.js}` containing `(runDiscoveryTest|runConnectTest|reachability|buildTree|dashboard/kpis)`.

F1. Business Logic in Components LOW

Benchmark: `Avg LOC per component = 74` → falls in the **Good** band (Good <150 · Moderate 150–300 · High Risk >300).

What to check: Validation, calculations, data transformation, or workflow logic inside view components.

Evidence: Pages are presentation + React Query orchestration. Heaviest files: `ConnectPage` 208 LOC, `DiscoveryPage` 162 LOC — still under 300 and mostly JSX. Domain KPI math is server-side. Residual workflow (invalidate queries after tests) lives in page handlers, acceptable for current size.

Evidence: Not observed as High Risk — components stay presentation-focused; avg 74 LOC across 9 UI files.

F2. Missing Frontend Service/Data Layer MEDIUM

Benchmark: `Components with inline API/data-access calls = 6` → falls in the **Good** band (Good <10 · Moderate 10–20 · High Risk >20). Priority Medium because paths are hard-coded despite a shared client.

What to check: `fetch` /HTTP calls and API URLs hard-coded inline in components instead of domain API modules.

Evidence: Shared `frontend/src/api/client.ts` exists (good), but pages embed path strings:

`frontend/src/pages/ConnectPage.tsx:28-45` :

```
const monitorsQuery = useQuery({
  queryKey: ['connect', 'monitors'],
  queryFn: () => api.get<{ data: ConnectMonitor[] }>('/connect/monitors'),
```

Same pattern in `DiscoveryPage` , `DashboardPage` , `LegacyDashboardWidget` , `LegacyMonitorPoller` , `MongoStatus` (6 components). No `discoveryApi.ts` / `connectApi.ts` facade.

Why it matters here: Endpoint renames (or Laravel vs Express path drift) require edits in every page. The shared client only centralizes `fetch` , not resource contracts.

Recommended approach: Add `frontend/src/api/discovery.ts` , `connect.ts` , `dashboard.ts` wrapping paths + types; pages call those only.

No files matched `frontend/src/**/*.tsx,ts,jsx,js` containing `api\.(get|post)<` .

F3. God / Oversized Components LOW

Benchmark: `Components >400 LOC = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single components with huge render + many state vars + side effects.

Evidence: Not observed — largest is `ConnectPage` at 208 LOC. `LegacyDashboardWidget` is multi-concern but small (59 LOC).

F4. Prop Drilling / Global State Abuse LOW

Benchmark: `Max prop-drilling depth = 2; Zustand store fields = 2 selection IDs` → falls in the **Good** band (Good ≤2 · Moderate 3–4 · High Risk >4).

What to check: Props threaded through many layers, or one giant global store.

Evidence: Not observed as abusive — `uiStore` only holds `selectedDiscoveryId` / `selectedMonitorId` . `LiveTestFeed` receives `events/isRunning/progress` one level deep. Server state is in React Query (appropriate).

F5. Legacy / Inconsistent Component Patterns MEDIUM

Benchmark: `Legacy-pattern components = 2` → falls in the **Moderate** band (Good 0 · Moderate 1–10 · High Risk >10).

What to check: Mixed paradigms (class + function), missing error boundaries, deprecated lifecycle/APIs.

Evidence:

`frontend/src/components/LegacyMonitorPoller.jsx` — class component with intentional missing unmount cleanup (interval leak):

```
componentDidMount() {
  this.intervalId = setInterval(() => {
    api.get<{ computed?: { reachability_pct: number } }>
      (`/connect/monitors/${this.props.monitorId}/checks`)
```

`frontend/src/pages/LegacyDashboardWidget.tsx:48` — throws on error with no Error Boundary in `App.tsx` :

```
if (error) throw new Error(error); // Uncaught – no Error Boundary wraps this component
```

Also mixed `.jsx` / `.tsx` extensions and unused legacy widget not routed from `App.tsx` (dead but still in tree).

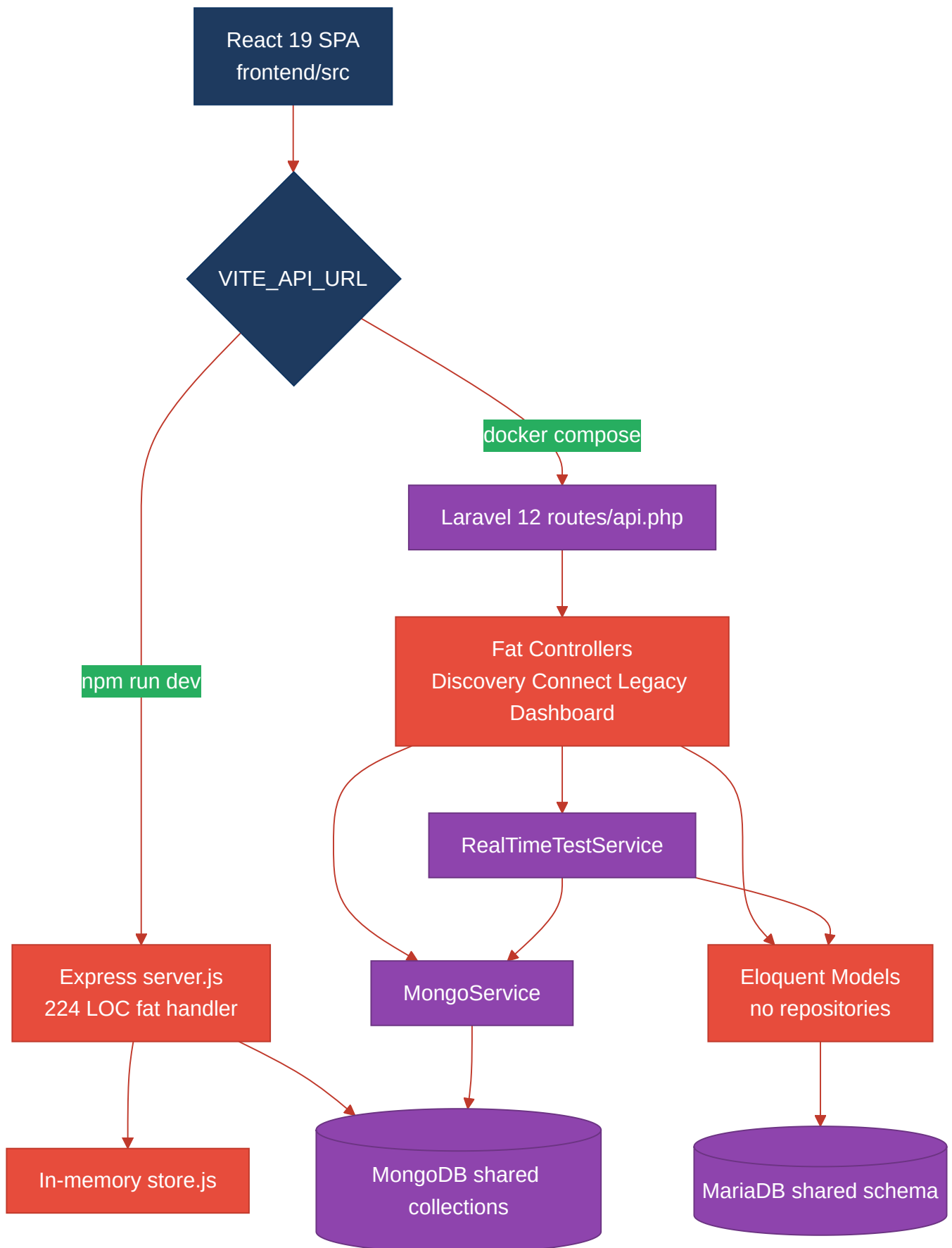
Why it matters here: New contributors copying `LegacyMonitorPoller` reintroduce class components and leaky intervals alongside the modern hooks pattern in `useRealtimeTest` .

Recommended approach: Rewrite poller as a hook; add a React Error Boundary around routes; delete or quarantine unused legacy widgets.

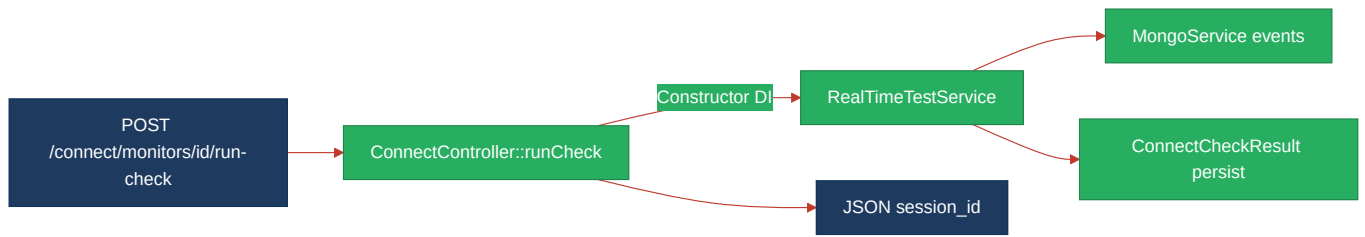
No files matched `frontend/src/**/*.{tsx,jsx,js,ts}` containing `(extends Component|componentDidMount|throw new Error)` .

1.3 Diagrams

Current-state architecture (as-is)

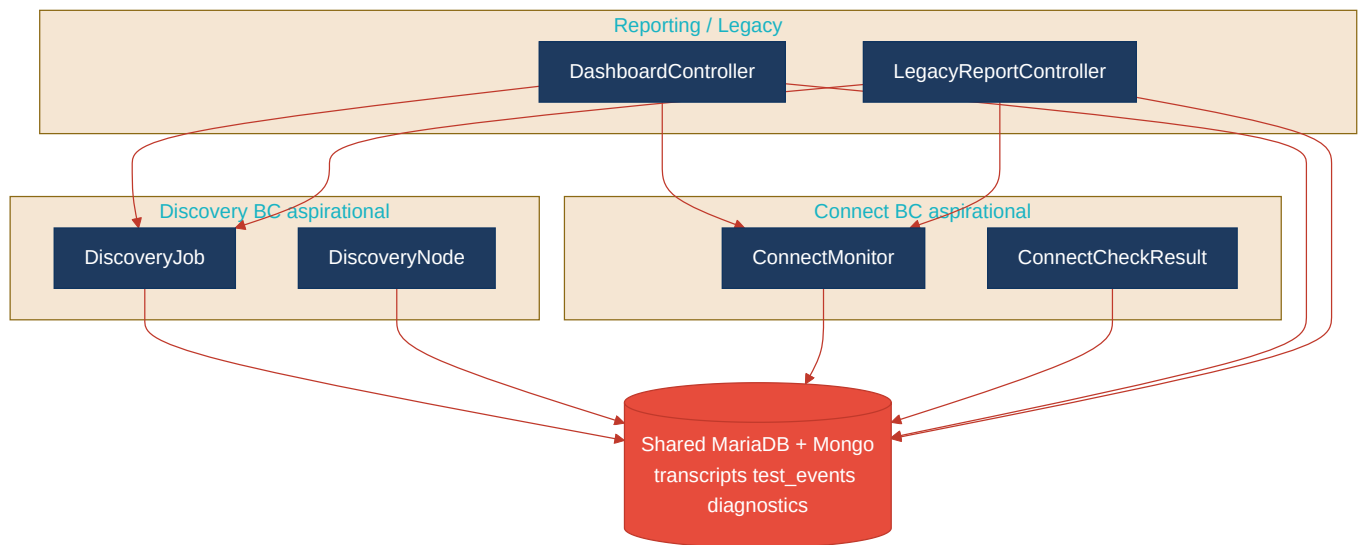


Clean reference path (target pattern found in codebase)

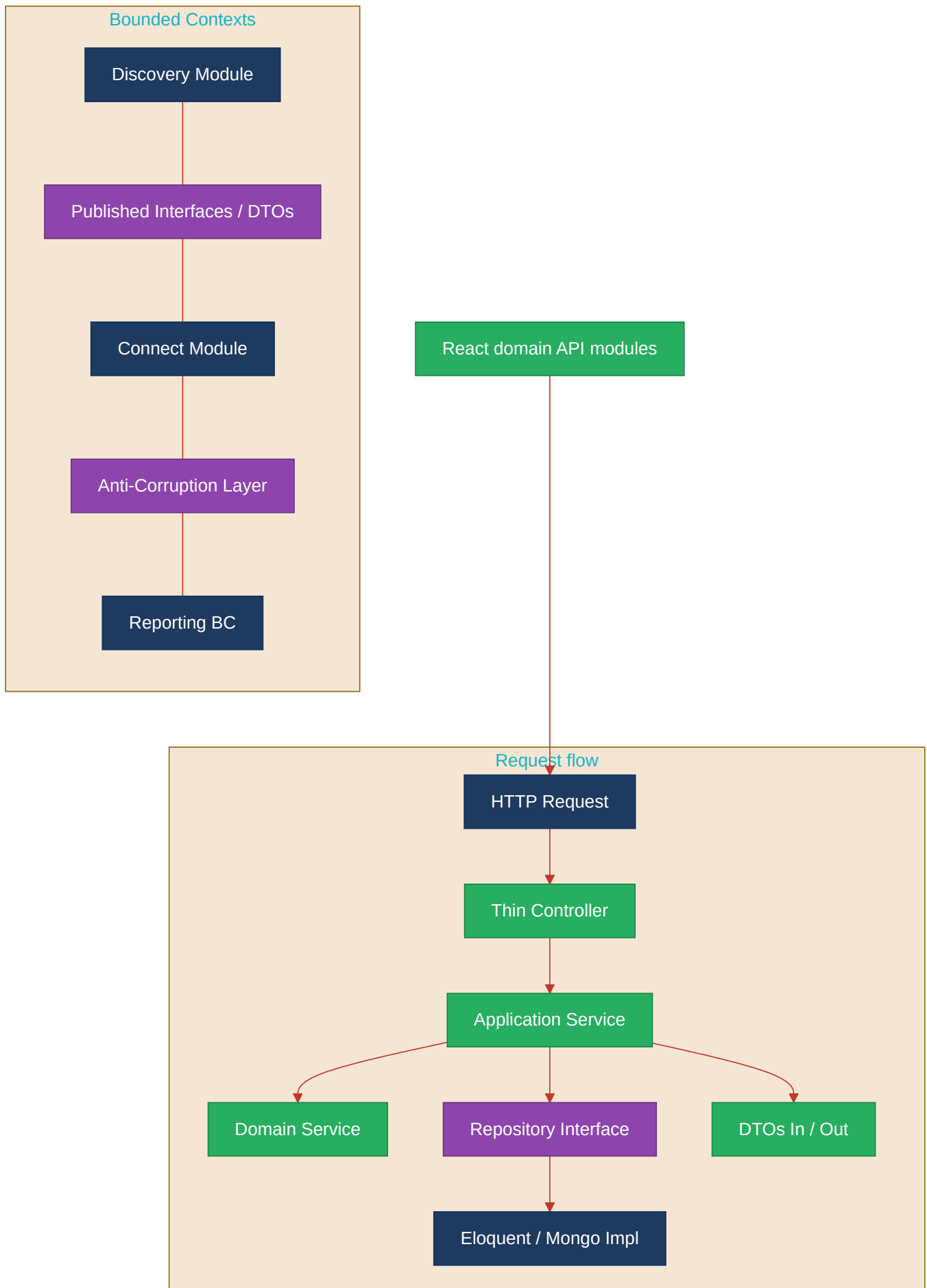


Thin DI path exists for test start (**ConnectController** / **DiscoveryController** → **RealTimeTestService**); CRUD/report paths do not follow it.

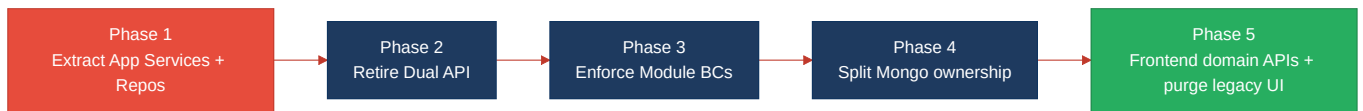
Domain boundary map (business domains found vs. shared data)



Target architecture (proposed)



Improvement roadmap



1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Extract reachability/tree/KPI logic from <code>ConnectController</code> , <code>LegacyReportController</code> , and Express <code>server.js</code> into Application Services	MODERATE	HIGH
H2 Missing Service Layer	Create <code>DiscoveryJobService</code> , <code>ConnectMonitorService</code> , <code>DashboardKpiService</code> ; stop Eloquent/store use in handlers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces for Eloquent models; bind in <code>AppServiceProvider</code> ; keep Mongo behind repository APIs	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> <code>extract()</code> with DTOs; single <code>IvrTreeBuilder</code> domain service	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Enforce repository-only persistence; remove Eloquent from <code>Http/Controllers</code> and Mongo from Express routes	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Populate real <code>Modules/Discovery</code> & <code>Modules/Connect</code> ; move Reporting behind published DTOs/ACL	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Split Mongo collections per BC; document MariaDB ownership; ban cross-module writes	HIGH RISK	CRITICAL
H10 Dual API Duplication	Consolidate on Laravel as single API; proxy or delete Express <code>dev-api</code> duplication	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Convert <code>LegacyMonitorPoller</code> to hooks; add Error Boundaries; remove unused legacy widget	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Application/Domain Services own workflows and are reusable from jobs/CLI.
- Repository interfaces enable testing without MariaDB/Mongo and isolate schema churn to one layer.
- Real bounded contexts (Discovery, Connect, Reporting) stop silent cross-module coupling and support future extraction.
- Eliminating the dual Laravel/Express stack removes the largest source of divergent business rules.
- Frontend domain API modules plus purged legacy components keep the SPA aligned with a single backend contract.